Task-1 Documentation(Deploy Node.js App on ECS Fargate with ECR)

1. Introduction

1.1 Purpose:-

The purpose of this document is to provide a comprehensive technical blueprint and record of the containerized deployment of the Node.js sample application. This document is intended for:

- **CTO:** To understand the high-level architecture and ROI of serverless containerization.
- **Developers:** For guidance on Dockerfile standards and application exposure.
- **Solution Architects:** To review the integration of AWS services (ECR, ECS, ALB).
- **DevOps Engineers:** For infrastructure management and troubleshooting.
- **Resellers:** To understand the packaged solution value.

1.2 Scope

This documentation covers the end-to-end DevOps process of containerizing a Node.js application, hosting the image in Amazon ECR, and orchestrating execution via AWS ECS Fargate with an Application Load Balancer. It does not cover external DNS registration (Route 53) or third-party CI/CD tools.

---

2. Overview of DevOps Project

2.1 Definition

This project implements a "Serverless Container" approach. By using DevOps principles, we automate the bridge between application code and cloud infrastructure, ensuring that environments are consistent and deployments are repeatable.

2.2 Key Principles (AWS Well-Architected Framework)

We achieved the 6 pillars through the following:

- **Operational Excellence:** Automated container health checks via ALB.
- **Security:** Isolation of tasks within private Security Groups.
- **Reliability:** Multi-AZ deployment across three subnets.
- **Performance Efficiency:** Using Fargate to match compute power precisely to demand.
- **Cost Optimization:** Pay-per-second billing with 256 vCPU and 512 MB RAM.
- **Sustainability:** Reducing idle energy by removing permanent EC2 instances.

2.3 Goals and Objectives:-

Migrate code to a production-ready containerized state.

- Provide a highly available endpoint via an ALB DNS.

2.4 Constraints and Assumptions * Constraints:-

Resources are limited to specific Fargate task sizes (0.25 vCPU).

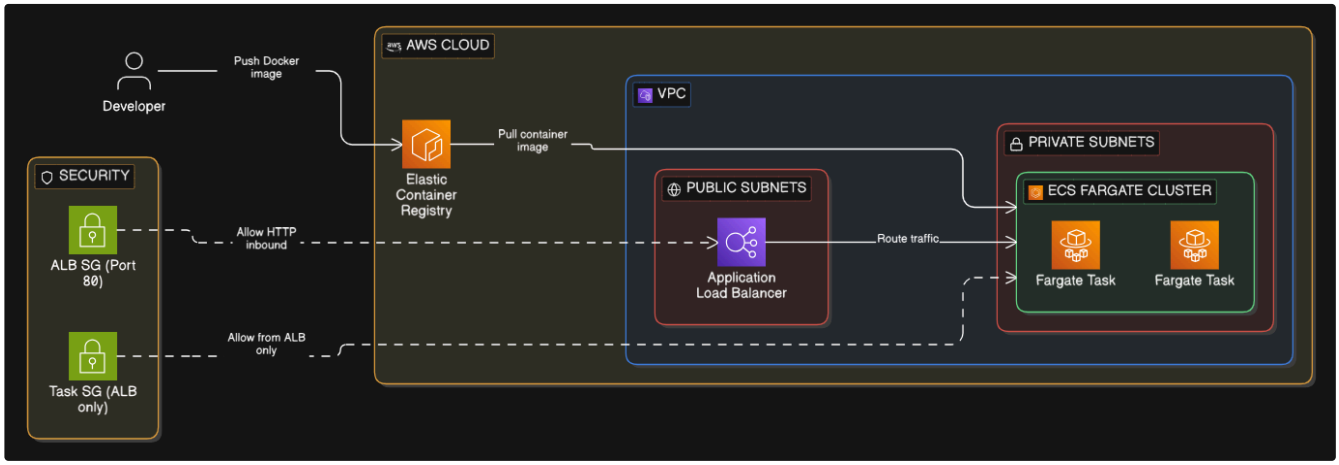
- **Assumptions:** The `node:24-alpine` base image remains available and secure.

---

3. Architecture and Infrastructure

3.1 Infrastructure Overview

The solution utilizes a VPC-based architecture where an Application Load Balancer acts as the entry point, distributing traffic to a cluster of Fargate containers.



3.2 Environments * Development:-

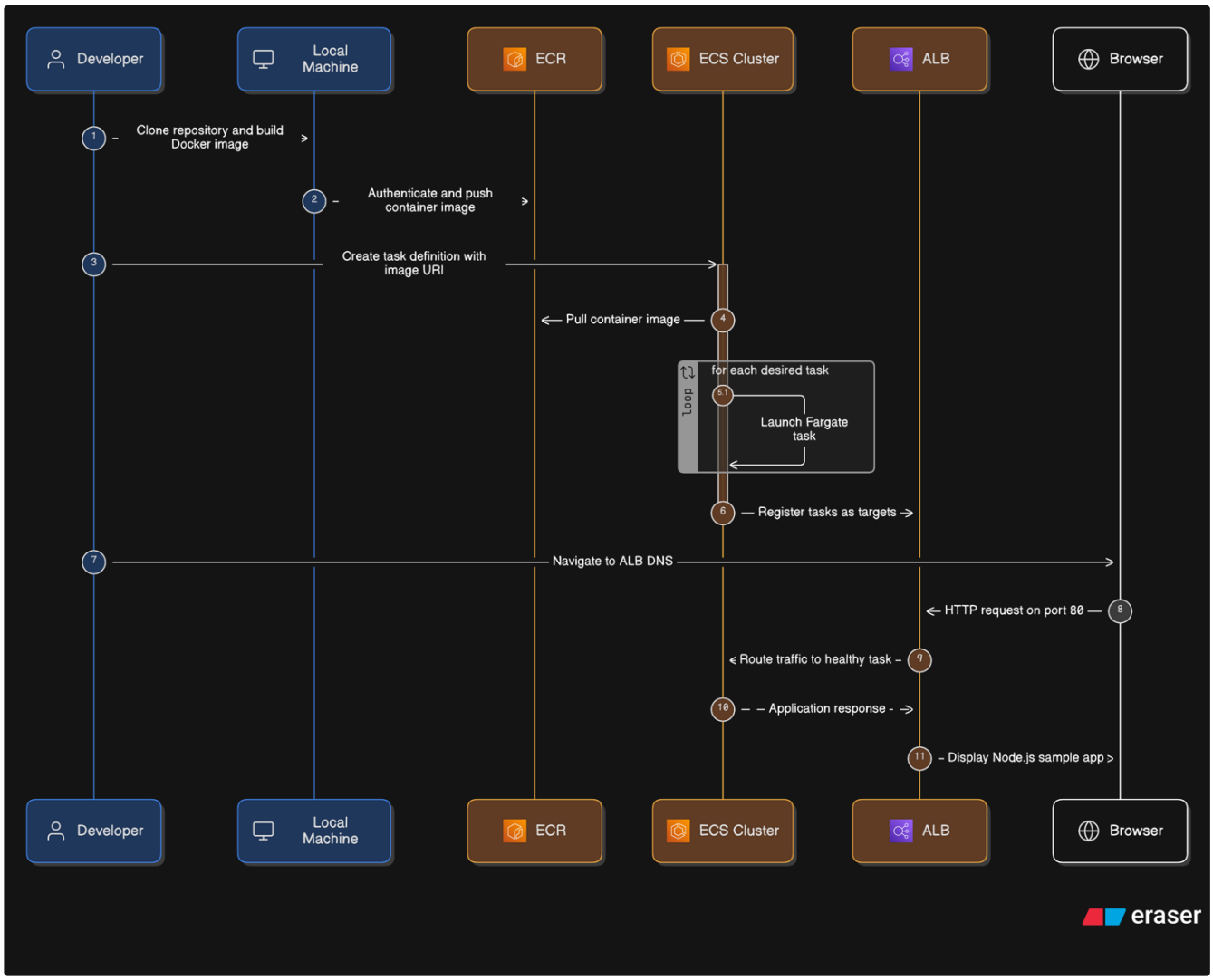
- **Registry:** Amazon ECR.
- **Orchestrator:** Amazon ECS (Fargate).
- **Networking:** VPC, ALB, Target Groups.

Environment	Purpose
Development	Local container testing
Staging	ECS validation
Production	Public ALB access

3.3 Tools and Technologies

Tool	Purpose
Docker	Containerization
Amazon ECR	Image storage
Amazon ECS Fargate	Container orchestration
Application Load Balancer	Traffic distribution
AWS CLI	Automation
GitHub	Source code repository

3.4 Application Architecture & User Request Flow

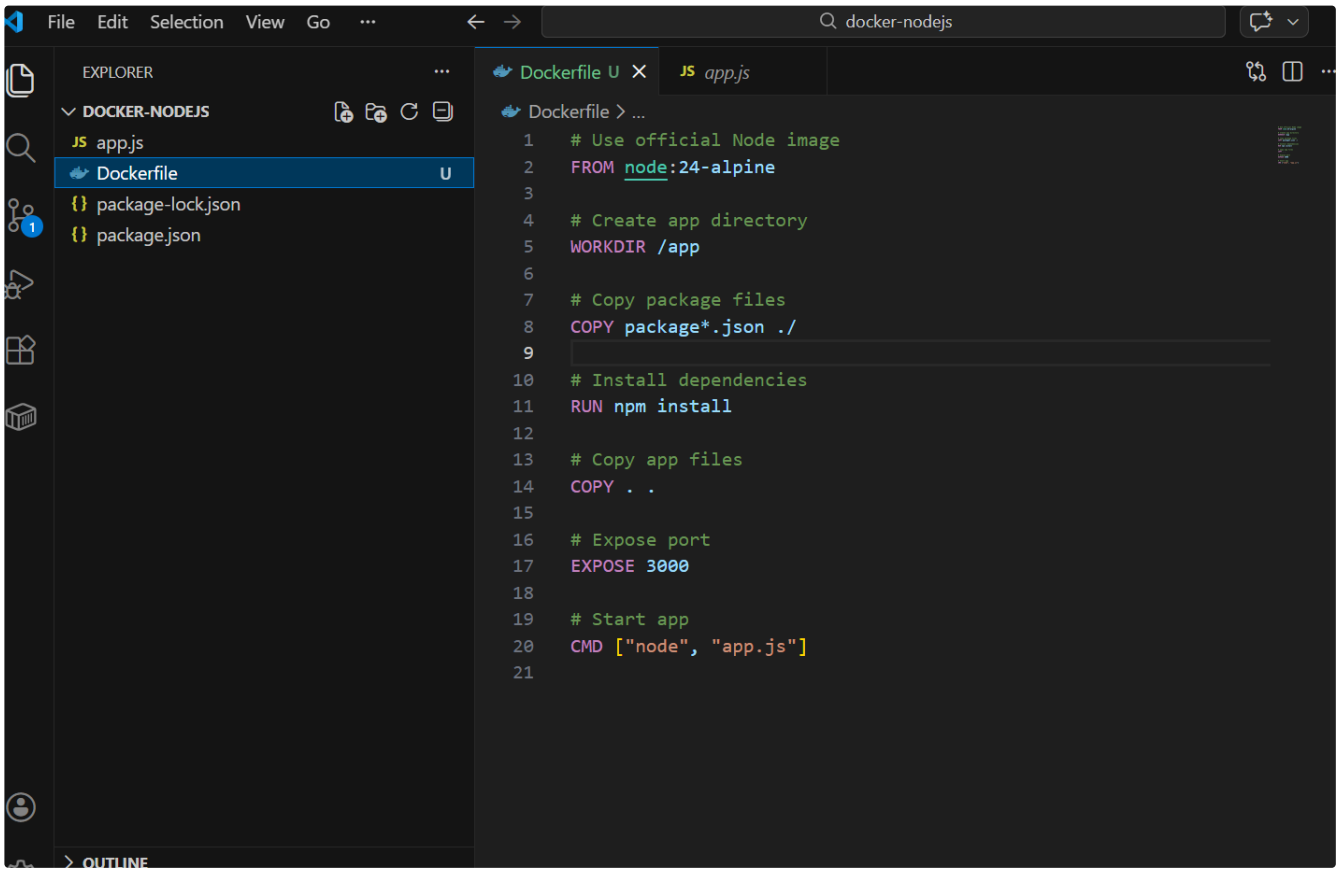


4. Development Practices

4.1 Version Control Code is managed via Git. The `docker-nodejs` repository serves as the single source of truth for the application and the `Dockerfile`.

4.2 Continuous Integration CI is initiated by building the Docker image locally. The `docker build` process ensures all dependencies are packaged before the image is pushed to the cloud.

4.3 Automated Testing Health checks are automated within the AWS Target Group. The ALB sends periodic pings to **Port 3000**; if a container fails to respond, it is marked unhealthy and replaced.



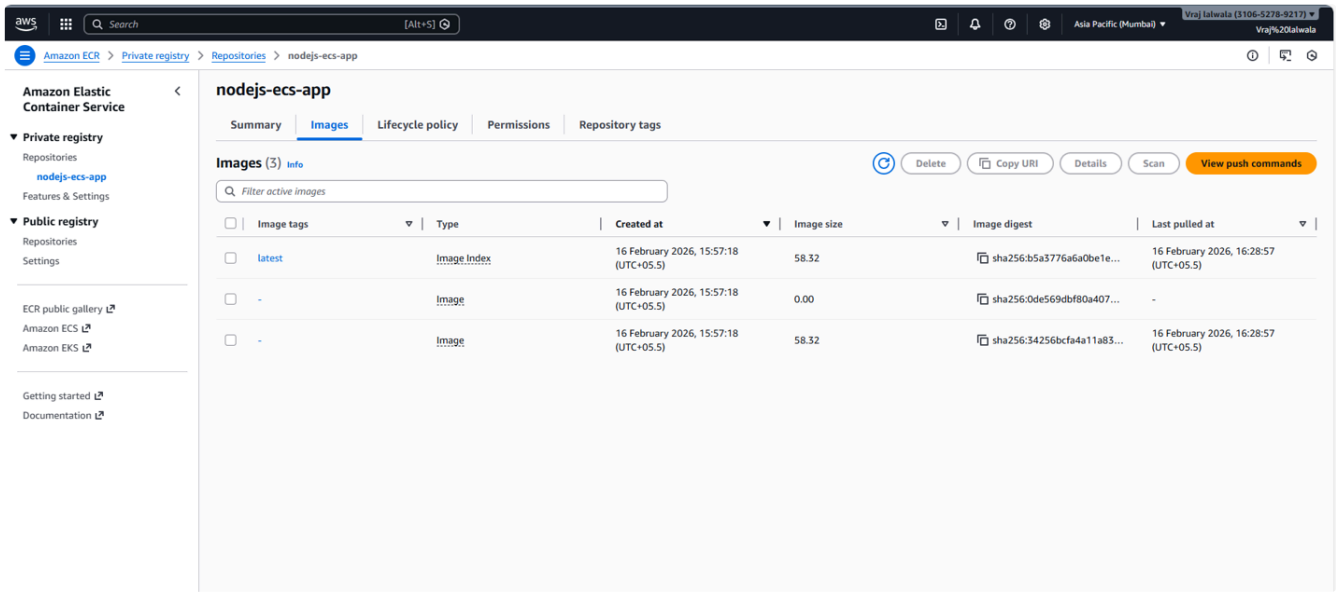
---

5. Deployment and Release Management

5.1 Deployment Strategy We utilize a **Rolling Update** strategy via ECS. When a new image is pushed, ECS launches new tasks and drains old ones only after the new versions pass health checks.

5.2 Configuration Management The **Task Definition** acts as the configuration-as-code template, defining the Image URI, memory limits, and port mappings (3000).

5.3 Continuous Delivery/Deployment Once the image is pushed to ECR with the `latest` tag, the ECS service is updated to pull the new version.



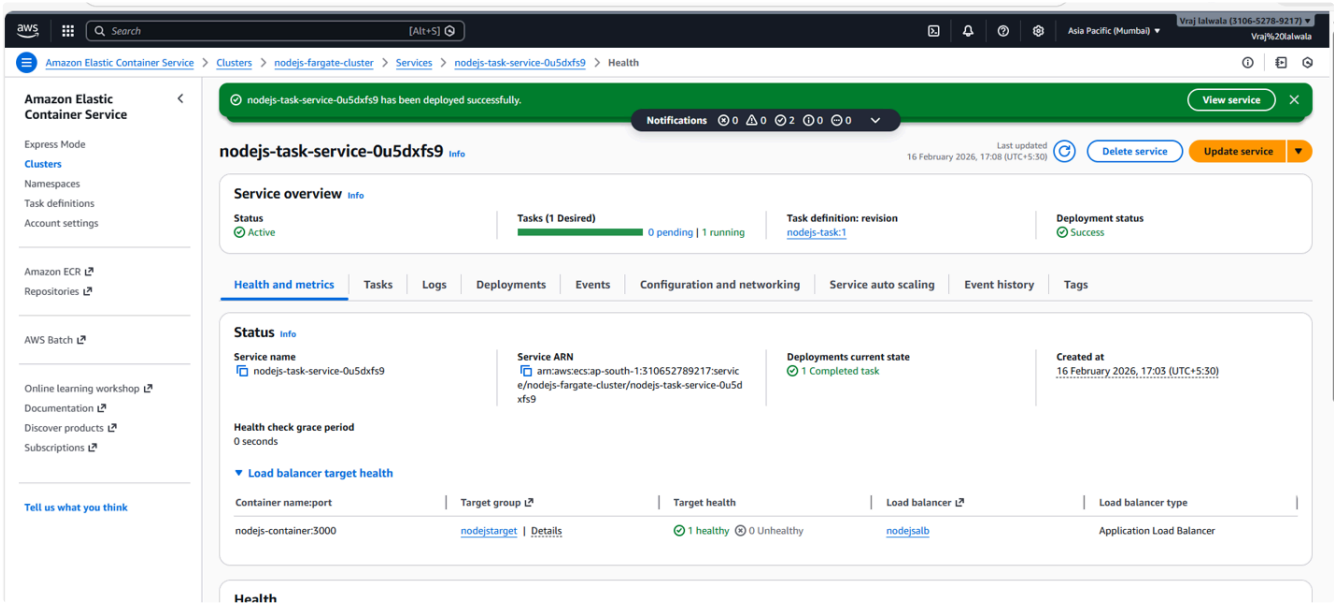
---

6. Monitoring and Incident Management

6.1 Monitoring Tools and Metrics AWS CloudWatch monitors CPU and Memory utilization. ECS provides real-time status on task health.

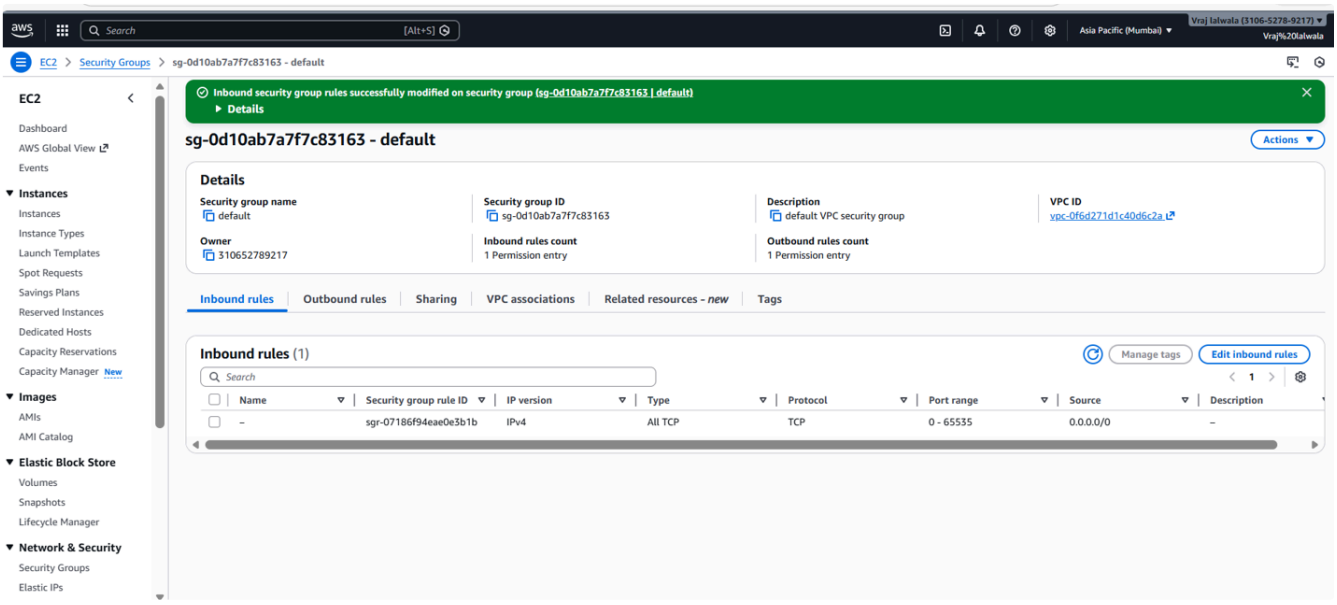
6.2 Alerting and Notification The ECS Service console provides immediate notification of deployment success or failure.

6.3 Incident Management If the "Running Count" drops below the "Desired Count," the ECS scheduler automatically triggers a new task replacement.



7. Security & Compliance

- **IAM:** The `AmazonEC2ContainerRegistryFullAccess` policy was used for initial setup, while the `ecsTaskExecutionRole` allows tasks to pull images.
- **Network:** Security Group `sg-0d10ab7a7f7c83163` restricts access.



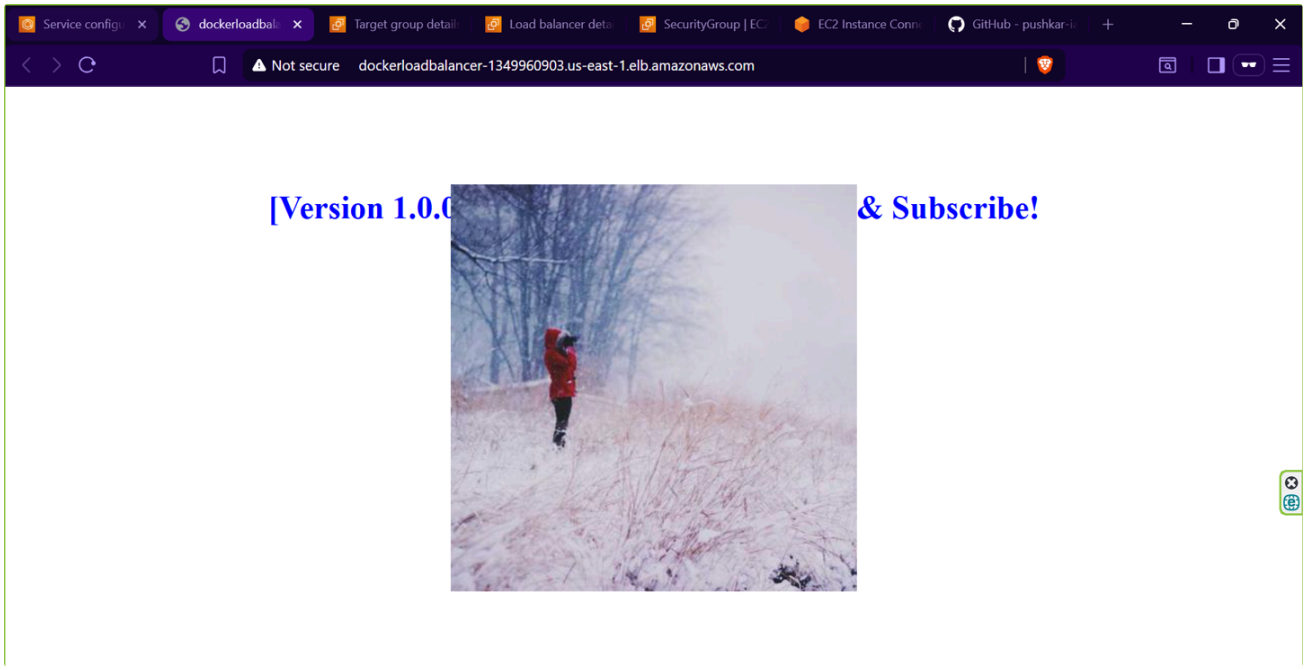
8. Cost Optimisation

The solution uses **Right Sizing** by selecting the smallest Fargate tier (256 CPU, 512 MB RAM). This ensures we do not over-provision for a sample application.

---

10. Implementation Timeline

- **Phase 1: Containerization (2 hours)** - Clone repo, write Dockerfile, build image.
- **Phase 2: Registry Setup (1 hour)** - Create ECR and push images.
- **Phase 3: Orchestration (3 hours)** - Cluster, Task Definition, and ALB setup.
- **Final Delivery:** February 16, 2026.



---

11. Continuous Improvement

Metrics from the ALB (Latency and Request Count) are reviewed to determine if task counts need to be increased (Autoscaling) in the future.

---

12. Conclusion

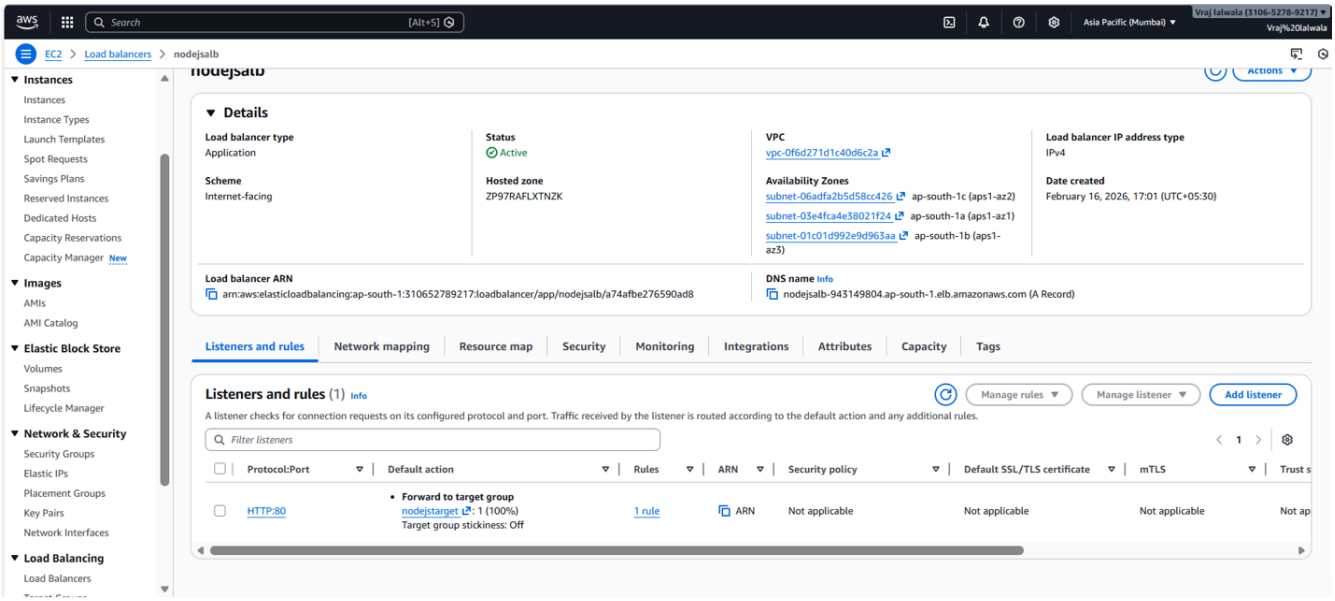
12.1 Summary The project successfully moved a Node.js app from local code to a cloud-native, serverless environment. This setup provides high availability, automatic scaling, and minimal management overhead.

12.2 Next Steps * Implement HTTPS via AWS Certificate Manager.

- Integrate a CI/CD pipeline for automated builds.

---

13. Customer Acknowledgement



Name: _____

Title: _____

Signature: _____

Date: _____